OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Studie zur verbesserten Anomalyerkennung mit erweiterter Kontextinformation

Research work
of

Yannic Hanel
3296954

curriculum
at the

Ostbayerischen Technischen Hochschule Regensburg
Faculty of Computer Science and Mathematics

Advisor: Prof. Dr. Sebastian Fischer

Regensburg, 20.02.2026

Contents

Abkürzungsverzeichnis **4**

1 Einleitung **1**
1.1 Motivation und Problemstellung 1
1.2 Zielsetzung der Arbeit 1
1.3 Aufbau der Arbeit 1

2 Related Work **3**

3 Systemarchitektur und Detection Framework **4**
3.1 Systemüberblick und verteilte Kommunikation 4
3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch 5
3.1.2 End-to-End-Datenfluss 5
3.2 Physische Testumgebung 7
3.3 Detection Einheiten der Ladeinfrastruktur 8
3.3.1 System- und Leistungsanomaliedetektion (pi_anomaly_detection) 9
3.3.2 Physikalische Signal- und Temperaturüberwachung (detection_power_temperature) 9
3.3.3 KI-basierte Angriffserkennung im Netzwerk (ids_power_ai_detection) . . 10
3.3.4 Hardware-Ereignisüberwachung (detection_hardwareevents) 10

4 Konzeption des Threat Reporters **11**
4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat Report 11
4.2 Zeitliche Diskretisierung und Intervalldefinition 11
4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse 12

5 Theoretische Grundlagen der Informationsfusion **13**
5.1 Vergleich von Aggregationsmethoden 13
5.1.1 Statische Aggregation 14
5.1.2 Padding und Masking 15
5.1.3 Deep Sets 16
5.1.4 Set2Set 16
5.1.5 Graph Neural Networks and Pooling 16

6 Methodik: Gewichtete Aggregation von Threat Reports **18**
6.1 Anforderungen an die Gewichtungsfunktion 18
6.2 Auswahl der Gewichtungsfunktion 18
6.2.1 Utility-Theory-Funktion 19
6.2.2 Sigmoid-Funktion 19
6.3 Implementierung des Attention-basierten Selektionsmodells 20

7 Evaluation **22**
7.1 Generierung der Threat Reports 22
7.2 Vergleich der Gewichtungsfunktionen 24
7.3 Echo Chamber Analyse 24

8 Diskussion und Forschungslücken **26**
8.1 Auswirkungen unterschiedlicher Gewichtungsstrategien auf die Report-Qualität . 26
8.2 Potenzial der LLM-gestützten Aufbereitung für externe CTI-Nutzung 26
8.3 Untersuchung des Einflusses von Report-Aggregation auf Detectionsergebnisse (Echo-Chamber-Effekt?) 26

9 Quellcode **27**

Bibliography **37**

Abkürzungsverzeichnis

DNN	Deep Neural Network
DoS	Denial of Service
IPFS	InterPlanetary File System
P2P	Peer-to-Peer
CID	Content Identifier
MADDR	Multiaddress
CTI	Cyber Threat Intelligence
I2C	Inter-Integrated Circuit
MSE	Mean Squared Error
LSTM	Long Short Term Memory
GAT	Graph Attention Networks
NaN	Not a Number

1 Einleitung

1.1 Motivation und Problemstellung

Obwohl die node-basierten Detektionsmechanismen der betrachteten Ladeinfrastruktur bereits eine beachtliche Erkennungsleistung aufweisen, besteht das Ziel, die Spezifität (True-Negative-Rate) sowie die Robustheit des Gesamtsystems weiter zu steigern. Lokale Anomalieerkennungsverfahren stoßen systembedingt an Grenzen, wenn netzwerkweite Veränderungen oder subtile, koordinierte Manipulationsversuche vorliegen, die auf Ebene einer einzelnen Wallbox nicht als bösartig identifiziert werden können.

Quelle evtl. mit meiner Bachelorarbeit arbeiten

Quelle schwierige erkennung wenn netzwerkweite Veränderung und nicht am Knoten selbst

Die Motivation dieser Arbeit liegt daher in der Entwicklung einer Methodik, die es einzelnen Ladepunkten ermöglicht, Kontextinformationen benachbarter Knoten in den Entscheidungsprozess einzubeziehen. Es wird vermutet, dass durch diesen kollaborativen Ansatz räumliche und zeitliche Korrelationen innerhalb des Netzwerks besser erfasst werden können, die über die isolierte Betrachtung eines einzelnen Knotens hinausgehen. Durch die Integration dieser Kontextdaten wird erhoft eine präzisere Differenzierung zwischen globalen Schwankungen und lokalen Sicherheitsvorfällen zu ermöglichen. Es wird erwartet, dass dieser erweiterte Kontextbezug zu einer Reduktion von Fehlalarmen (False Positives) sowie einer verbesserten Erkennungsrate tatsächlicher Bedrohungen (True Positives) beiträgt. Inwiefern diese Annahmen empirisch belastbar sind, bleibt jedoch offen und soll in einer weiterführenden Arbeit durch die Evaluation anhand eines unabhängigen Datensatzes systematisch untersucht werden.

1.2 Zielsetzung der Arbeit

Um die dargelegte Problemstellung zu adressieren, liegt der Fokus dieser Arbeit auf der Konzeption und Implementierung einer robusten Infrastruktur für die Anomalieerkennung innerhalb der Ladeinfrastruktur. Übergeordnetes Ziel ist die Schaffung einer stabilen Betriebsumgebung, die einen zuverlässigen, kontextbasierten Datenaustausch zwischen den einzelnen Knoten gewährleistet.

Ein wesentlicher Aspekt hierbei ist die Realisierung einer konsistenten und effizienten Aggregation dezentraler Informationen, um die Detektionsgüte der beteiligten Entitäten signifikant zu steigern. Nur durch eine performante Umgebung, welche den reibungslosen Austausch von Kontextinformationen sicherstellt, kann die in Kapitel 3 (Systemarchitektur und Detection Framework) detaillierte Anomalieerkennung erfolgreich implementiert und langfristig wartbar betrieben werden. Die Entwicklung dieser stabilen Kommunikations- und Verarbeitungsplattform bildet somit den zentralen Kernbestandteil der vorliegenden Arbeit.

1.3 Aufbau der Arbeit

Zunächst wird die Systemarchitektur im zur Verfügung stehenden Peer-to-Peer-Netzwerk beschrieben sowie die implementierten Detektionen und die Datenakquise über Prometheus empirisch erläutert und differenziert.

Darauf folgend wird die Konzeption des Threat Reporters vorgestellt, welcher die Detektionsergebnisse in einem standardisierten Format aggregiert und über definierte Schnittstellen zwischen den Knoten überträgt.

Anschließend werden die theoretischen Grundlagen der Informationsfusion präsentiert.

Darauf folgend wird die Methodik der gewichteten Aggregation mittels Utility Theory beschrieben.

Anschließend wird die erweiterte Architektur und umsetzung der Vorgestellten Methodik beschrieben. Die Daraus generierten Threat Reports werden präsentiert und hinsichtlich ihrer Qualität und Nützlichkeit für die interne Nutzung der Knoten bewertet.

Die anschließende Diskussion der Ergebnisse führt zur Identifikation einer Forschungslücke, welche in zukünftigen Arbeiten weiter untersucht werden sollte.

Vielleicht

TODO: Sollte hier wissenschaftlich begründet werden, warum die Wahl auf eine Kombination aus Deep Sets und Attention-Mechanismen gefallen ist?

TODO: Die Formel der Utility Theory wurde nicht originär für Gewichtungsfunktionen entwickelt. Wie problematisch ist das? Sollte dies in der Diskussion thematisiert oder direkt eine alternative Methode verwendet werden?

TODO: Fokus darauf legen wie die zuvor beschriebenen Methoden in die Architektur einbepflegt wurden

Welche Forschungslück

TODO: Mögliche Forschungslück

2 Related Work

3 Systemarchitektur und Detection Framework

3.1 Systemüberblick und verteilte Kommunikation

Das ReSiLENT-System gliedert sich in zwei logische Domänen: die **Charging Station** mit dem eingebetteten Detection-Client sowie das **Cloud Dashboard** zur zentralen Visualisierung und Auswertung. Beide Domänen sind über ein privates Peer-to-Peer-Netzwerk auf Basis des *Inter-Planetary File System* (IPFS) lose gekoppelt. Anstelle direkter IP-zu-IP-Verbindungen erfolgt der Datenaustausch über inhaltsadressierte Bezeichner (*Content Identifier*, CIDs).

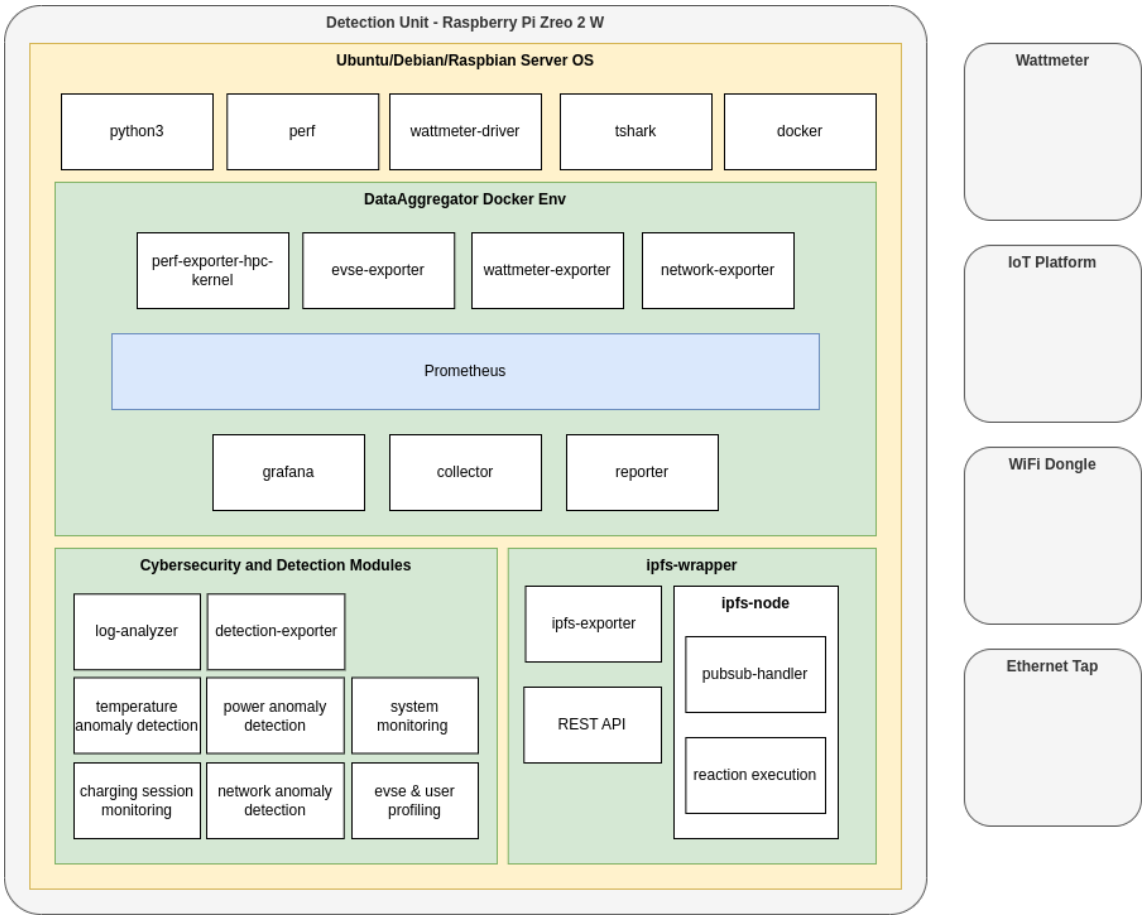

Figure 1: Gesamtarchitektur des ReSiLENT-Systems mit Charging Station und Cloud Dashboard

Abbildung 1 zeigt die Gesamtarchitektur des Systems mit Fokus auf die in jeder Wallbox eingebettete Detection Unit. Als zentrales Laufzeitmodul fungiert der *DataAggregator*, der die erfassten Sensordaten der Wallbox über Prometheus bereitstellt und sowohl den Cybersecurity- und Detection-Modulen als auch Werkzeugen zur Datenaufbereitung zugänglich macht.

Die Architektur ist auf kollaborative Angriffserkennung ausgelegt: Über den *ipfs-wrapper* und dessen **Publish-Subscribe-Mechanismus** (detailliert in Abbildung 2) teilen Ladesäulen Kontextinformationen mit allen Netzwerkteilnehmern und können somit die situative Wahrnehmung über die gesamte Ladeinfrastruktur hinweg verbessern.

3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch

Für die verteilte Kommunikation zwischen den Detection Units und dem Cloud Dashboard wird ein **privates IPFS-Netzwerk** eingesetzt. Im Gegensatz zum öffentlichen IPFS-Netz wird der Zugang durch einen gemeinsamen *Swarm Key* beschränkt, sodass ausschließlich autorisierte Knoten dem Netzwerk beitreten können. Die Netzwerktopologie ist in Abbildung 2 dargestellt.

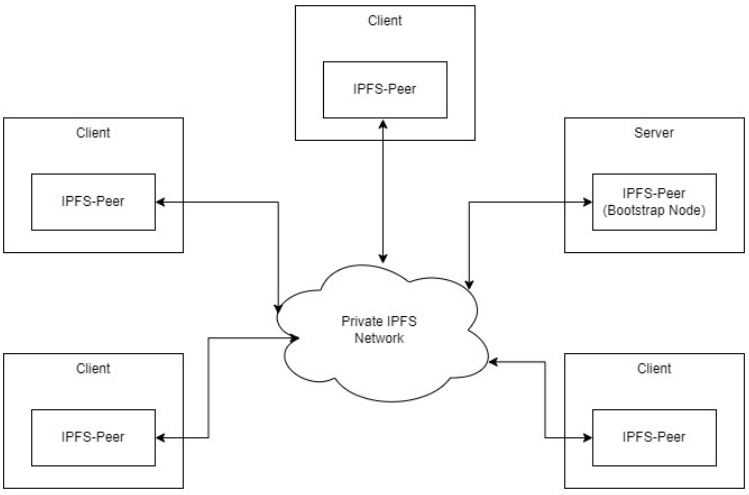

Figure 2: Topologie des privaten IPFS-Netzwerks mit Bootstrap-Node und Client-Peers

Der Netzbeitritt erfolgt über einen dedizierten **Bootstrap-Node**, der auf dem Server des Cloud Dashboards betrieben wird. Neue Peers melden sich beim Bootstrap-Node an, erhalten die Multiaddresses (*MADDR*) der bekannten Peers und kommunizieren anschließend direkt miteinander, ohne den Bootstrap-Node als Relay zu verwenden. Jeder Knoten betreibt einen lokalen **IPFS-Peer-Daemon**, der über den `ipfs-wrapper`, wie in Abbildung 3 dargestellt, in die Systemarchitektur eingebunden ist.

Für die Benachrichtigung über neue Daten wird der **Publish-Subscribe-Mechanismus** von IPFS genutzt: Sobald neue Messdaten erhoben werden, können diese veröffentlicht werden indem sie die zugehörige CID zusammen mit der eigenen MADDR über einen gemeinsam abonnierten Kanal an alle Teilnehmer versendet. Die Empfänger rufen die Datei daraufhin direkt beim Ursprungs-Peer ab. Da eine CID dem kryptografischen Hash des jeweiligen Inhalts entspricht, ist die Datenintegrität implizit gewährleistet, ohne dass zusätzliche Signaturmechanismen erforderlich sind.

3.1.2 End-to-End-Datenfluss

Der vollständige Datenfluss vom Sensor bis zur Visualisierung lässt sich in drei Phasen unterteilen: Datenerhebung und Aggregation auf der Charging Station, Transport über das IPFS-Netz sowie Persistierung und Darstellung im Cloud Dashboard. Das Sequenzdiagramm in Abbildung 4 sowie der Datenfluss in Abbildung 3 illustrieren diesen Ablauf.

Datenerhebung (Charging Station)

Der *Aggregator* innerhalb des Knotens sammelt die Messdaten. Prometheus scraped alle Exporter in konfigurierbaren Intervallen. Der *Reporter* liest die aggregierten Daten aus der Prometheus-

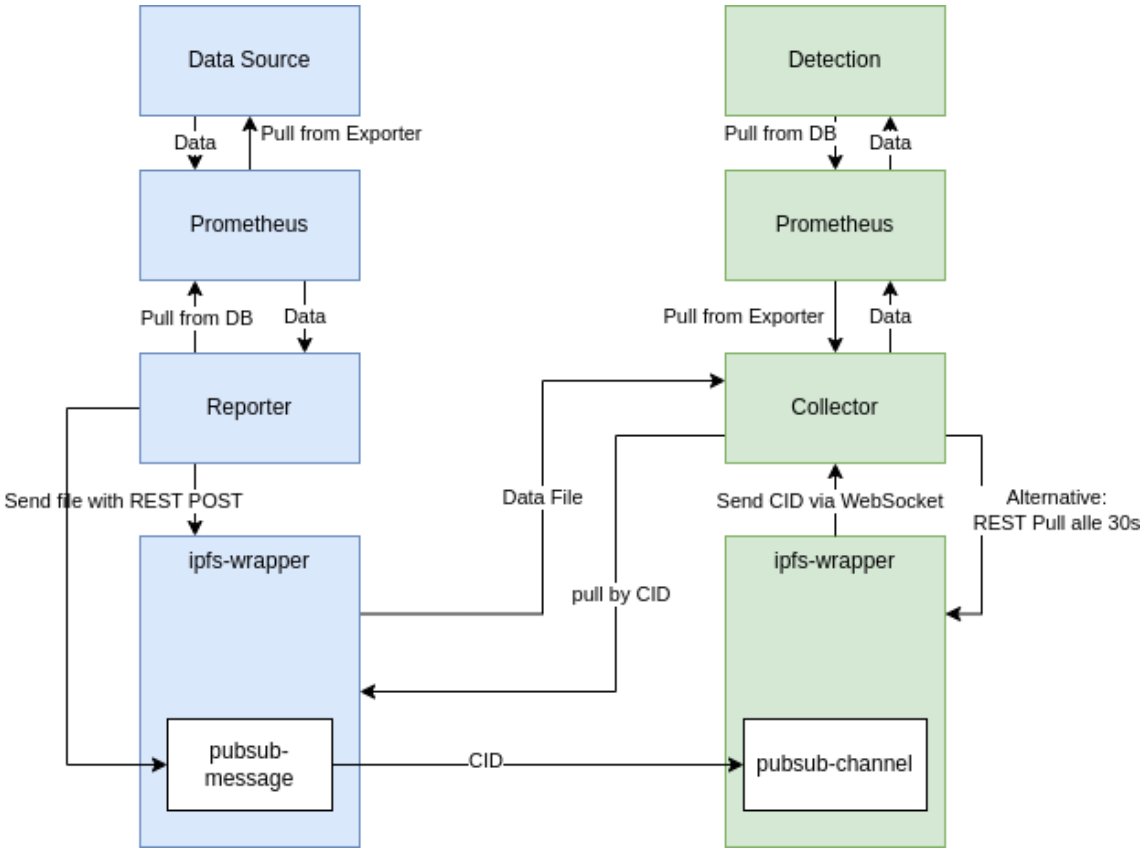

Figure 3: Datenfluss zwischen Charging Station (blau) und Cloud Dashboard (grün) über IPFS

Datenbank und übermittelt sie an den `ipfs-wrapper`.

Transport (IPFS-Netzwerk)

Der `ipfs-wrapper` nimmt die Datei entgegen, publiziert sie im lokalen IPFS-Node und erhält eine CID zurück. Anschließend wird die CID zusammen mit der eigenen MADDR über den konfigurierten **Publish-Subscribe-Mechanismus** im privaten IPFS-Netz verbreitet. Die eigentlichen Messdaten werden dabei nicht aktiv gepusht, lediglich der Zeiger (CID) wird distribuiert.

Persistierung und Visualisierung (Cloud Dashboard)

Auf der Serverseite empfängt der *Publish-Subscribe-Event-Listener* des `ipfs-wrapper` die CID und MADDR und leitet diese per **WebSocket** an den Collector weiter. Der Collector fordert daraufhin die vollständige Datei über den lokalen IPFS-Peer an, der sie in Chunks vom Ursprungs-Peer abruft. Der *File Extractor* verarbeitet die Rohdaten und persistiert die Messwerte in **InfluxDB**. Grafana greift auf InfluxDB zu und stellt die Zeitreihendaten im Dashboard dar (vgl. Abbildung 5).

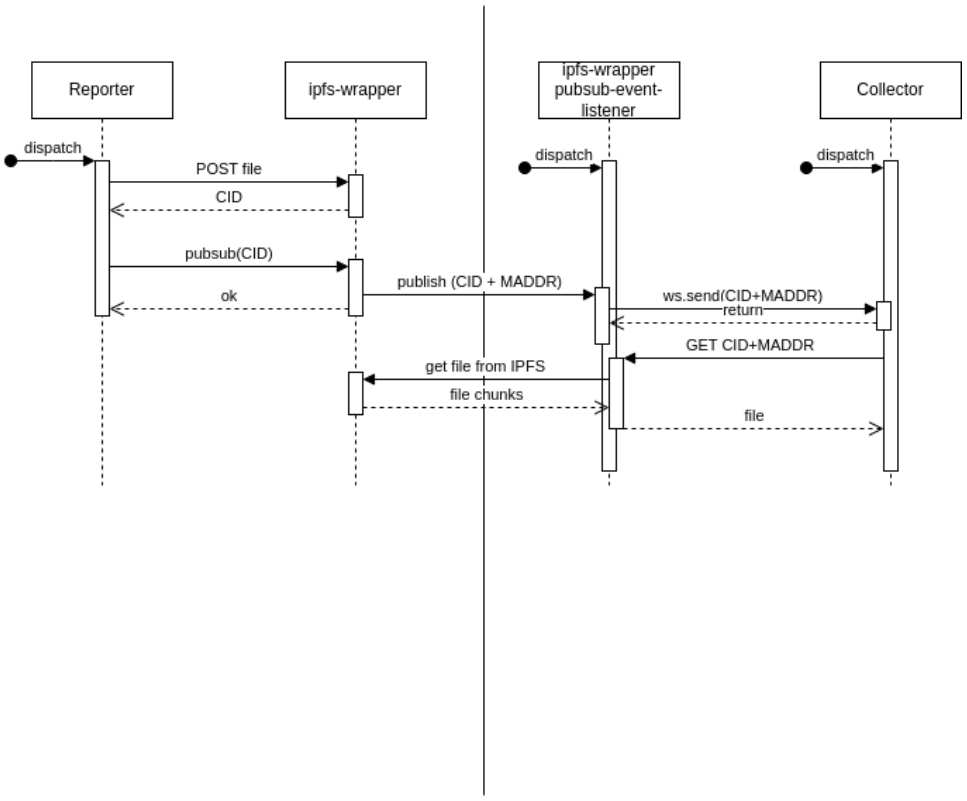

Figure 4: Sequenzdiagramm des IPFS-basierten Datenaustausches

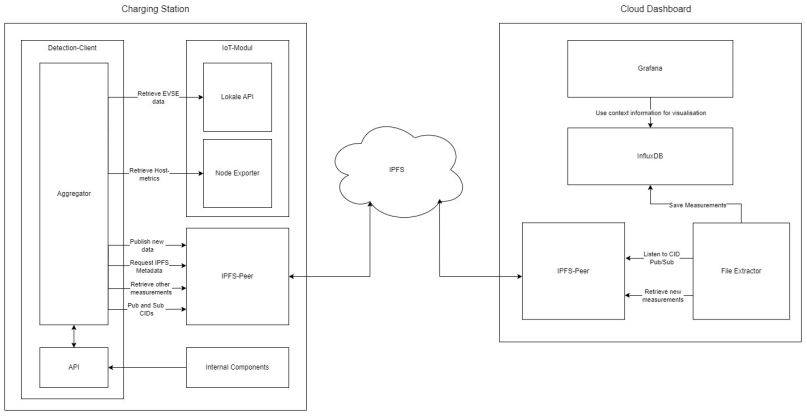

Figure 5: Komponenten des Cloud Dashboards: IPFS-Peer, File Extractor, InfluxDB und Grafana

Dieses Verständnis der Systemarchitektur und des Datenflusses bildet die Grundlage für die folgende Arbeit zur Implementierung des CTI-Reports und später des aggregierten CTI-Reports.

3.2 Physische Testumgebung

Die physische Testumgebung für die Implementierung und Evaluierung der vorliegenden Arbeit besteht aus einer realen ChargeIQ-Wallbox. Die Detection-Module sind in diesem Szenario abweichend von der in Kapitel "Systemüberblick und verteilte Kommunikation" 3.1 beschriebenen

Zielarchitektur nicht auf einem Raspberry Pi Zero 2 W, sondern zu Testzwecken auf einem leistungsfähigeren Raspberry Pi 4 B oder BananaPi M5 installiert. Diese sind über eine kabelgebundene Ethernet-Verbindung in das private IPFS-Netzwerk eingebunden, um eine stabile und performante Kommunikation zu gewährleisten. Die Wallbox selbst ist ebenfalls über eine drahtlose Verbindung in dasselbe private IPFS-Netzwerk integriert.

Darüber hinaus besteht eine serielle *I2C*-Verbindung (*Inter-Integrated Circuit*) zwischen der ChargeIQ-Wallbox und einem Einplatinencomputer in diesem fall der Raspberry Pi 4 B, welcher stellvertretend für die spätere Detection Unit auf Basis des Raspberry Pi Zero 2 W fungiert. Diese Verbindung ermöglicht die Erfassung von Echtzeitdaten des integrierten Wattmeters, welcher die sensor daten für eine im Kapitel "Detection Units der Ladeinfrastruktur" **??** beschriebene Detektionseinheit bereitstellt.

Zusätzliche Sensordaten werden dem Raspberry Pi 4 B über die in der Testumgebung eingerichtete drahtlose WLAN-Verbindung übermittelt, in welche sowohl die als Detection Unit eingesetzte Einplatinencomputer als auch die Wallboxen eingebunden sind.

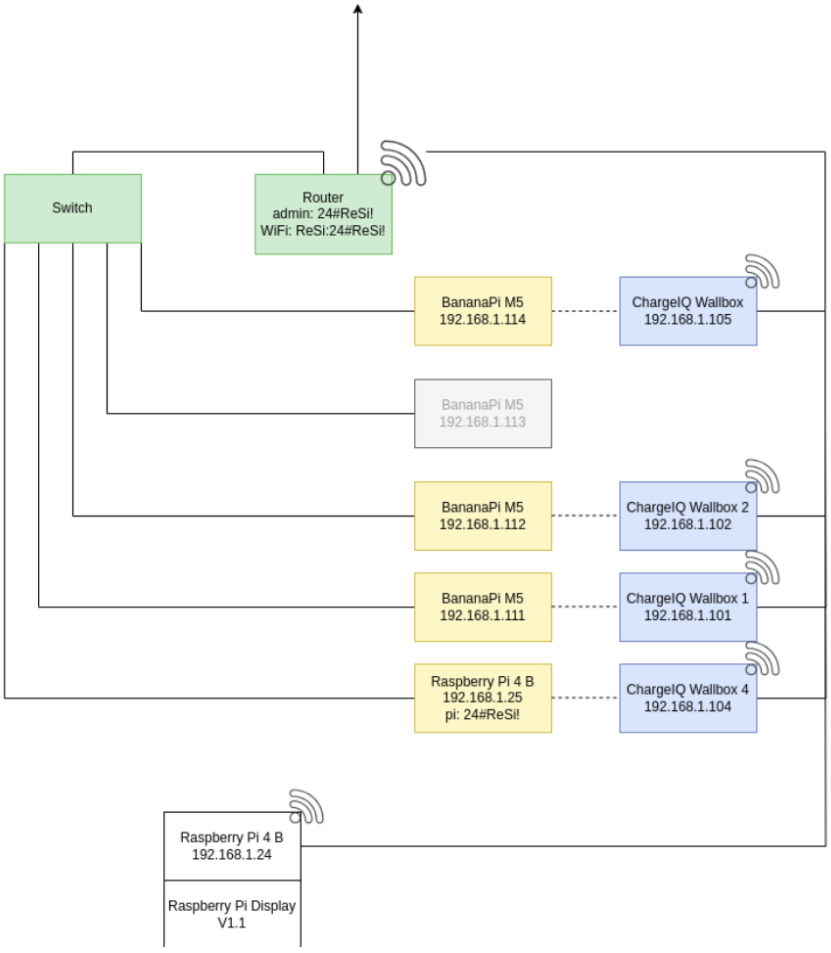

Figure 6: Schematische Darstellung der physischen Testumgebung.

3.3 Detection Einheiten der Ladeinfrastruktur

Im Folgenden werden die vier Hauptdetektionseinheiten der Testumgebung der Ladeinfrastruktur vorgestellt, die im Rahmen dieses Projekts implementiert wurden und die für den Cyber Threat

Report sowie den Aggregierten Cyber Threat Report von zentraler Bedeutung sind.

3.3.1 System- und Leistungsanomaliedetektion (pi_anomaly_detection)

Die *Pi Anomaly Detection Einheit* trifft Entscheidungen anhand der folgenden Metriken: Shunt-Spannung, Bus-Spannung, Stromstärke und Leistung. Diese Messdaten werden von den Sensoren der Wallbox erfasst und an Prometheus weitergeleitet. Die beschriebene Anomaliedetektionseinheit greift diese Daten über Prometheus auf und nutzt daraufhin zwei Deep-Learning-Modelle:

- **xLSTM-Autoencoder:** Dieses Modell operiert auf einem Sliding-Window von $w = 44$ Sekunden. Es lernt die Normalverteilung der physikalischen Parameter (Spannung, Strom, Leistung). Eine Anomalie wird detektiert, wenn der Rekonstruktionsfehler E_t den Schwellenwert $\tau = 0.008$ überschreitet. Die Wahrscheinlichkeit wird über eine Sigmoid-Funktion approximiert:
$$P_{xlstm} = \frac{1}{1 + e^{-50 \cdot (E_t - \tau)}}$$
- **Transformer-Klassifikator:** Parallel dazu extrahiert das System mittels *tsfresh* 15 statistische Merkmale aus einem 60-Sekunden-Fenster. Ein Transformer-Modell klassifiziert diese aggregierten Merkmale direkt in die Kategorien *benign* oder *attack*.
- **Sequentielles Voting-Verfahren:** Zur Stabilisierung der Detektion nutzt das System einen *Deques*-Puffer der Länge $k = 3$. Ein Alarm wird nur ausgelöst, wenn:
  - Beide Modelle im aktuellsten Zeitschritt eine Anomalie melden ODER
  - Mindestens eines der Modelle eine konsistente Sequenz von drei aufeinanderfolgenden positiven Detektionen aufweist.

Die Detektionseinheit reagiert insbesondere auf Stromspitzen und -einbrüche, welche primär über die genannten Metriken identifiziert werden können.

3.3.2 Physikalische Signal- und Temperaturüberwachung (detection_power_temperature)

Diese Detektionseinheit arbeitet mit einer klassischen regelbasierten Logik, welche IDS-Muster zuordnet und auf Basis von Schwellenwerten für physikalische Parameter (Spannung, Leistung, Temperatur) Anomalien detektiert. Die Detektion ist in zwei Bereiche der IDS-Mustererkennung unterteilt:

- **Temperaturüberwachung:** Diese Anomalieerkennung ruft die Temperaturdaten der letzten 60 Sekunden von Prometheus ab und erkennt Anomalien anhand von drei definierten Regeln:
  - *Absolute Obergrenze:* Wenn die Temperatur im abgefragten Zeitfenster einen Wert von 70 °C überschreitet, wird eine Anomalie detektiert.
  - *Konstantreihe + Sprung:* Wenn ein Temperatursprung bei einer ansonsten konstanten Reihe eine Toleranz von ± 1 überschreitet, wird eine Anomalie gemeldet.
  - *Anomalie durch Z-Score:* Wenn die Standardabweichung der Temperaturwerte einen Z-Score von $> 3,0$ erreicht, löst die Anomalieerkennung aus.

- **Wattmeter-Überwachung:** Analog zur System- und Leistungsanomaliedetektion (siehe Abschnitt 3.3.1) ruft diese Detection ebenfalls vier Metriken zu Spannung und Leistung der letzten 60 Sekunden ab. Die Detektion prüft, ob diese Werte fest definierte Schwellenwerte (Minimum oder Maximum) unter- bzw. überschreiten. Ist dies der Fall, wird eine Anomalie ausgelöst.

Sobald die Temperaturüberwachung und/oder die Wattmeterüberwachung eine positive Anomalie detektieren, schlägt auch der kombinierte Alarm *detection_power_temperature* aus.

3.3.3 KI-basierte Angriffserkennung im Netzwerk (ids_power_ai_detection)

Diese Detection-Komponente analysiert dieselben vier physikalischen Messgrößen, die in Abschnitt 3.3.1 beschrieben wurden, verzichtet jedoch vollständig auf statische Schwellenwerte. Stattdessen kommen zwei komplementäre Machine-Learning-Modelle zum Einsatz, die anomales Verhalten im zeitlichen Kontext erkennen:

- *Isolation Forest:* Dies ist ein Verfahren, welches einzelne Messpunkte anhand ihrer statistischen Isolation im Merkmalsraum bewertet. Für jeden Messzeitpunkt wird ein Anomalie-Score berechnet. Weicht ein Datenpunkt signifikant vom gelernten Normalzustand ab, wird dies als Anomalie gewertet.
- *LSTM-Autoencoder:* Wie für LSTM-Modelle (Long Short-Term Memory) üblich, betrachtet dieses Modell nicht nur einzelne Messwerte, sondern eine Sequenz von 60 Sekunden (bei einem Abfrageintervall von 5 Sekunden). Das Modell wurde auf normalem Betriebsverhalten trainiert und lernt, typische Zeitreihenmuster zu rekonstruieren. Zur Erkennung wird der Mean Squared Error (MSE) zwischen der originalen Sequenz und ihrer Rekonstruktion berechnet. Überschreitet der MSE des aktuellen Fensters einen empirisch bestimmten Schwellenwert (`LSTM_THRESHOLD` = 0,01), wird eine Anomalie ausgelöst.

3.3.4 Hardware-Ereignisüberwachung (detection_hardwareevents)

In diesem Verfahren werden knapp 20 relevante Hardware-Ereignisse kontinuierlich bewertet und mittels eines LSTM-Modells auf anomales Verhalten geprüft.

Das Modell analysiert die Daten in einem gleitenden Zeitfenster. Ergibt die Modellvorhersage eine Angriffswahrscheinlichkeit von über 50 %, wird das Ereignis als Anomalie klassifiziert. Die Ergebnisse, einschließlich der Einzelwahrscheinlichkeiten für *benign* und *attack*, werden über eine dedizierte Schnittstelle an Prometheus exportiert und zur Verfügung gestellt.

4 Konzeption des Threat Reporters

4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat Report

Um die Interoperabilität und Standardisierung des Threat Reports zu gewährleisten, wurde entschieden, die erforderlichen Daten zentral aus der Lokalen Prometheus-Datenbank zu extrahieren, anstatt diese einzeln von den jeweiligen Detection-Einheiten abzurufen. Hierfür wurden spezifische Prometheus-Exporter für die Detektionseinheiten entworfen und integriert. Diese stellen die Daten kontinuierlich über die lokalen Ports 2112, 2113, 2115 und 2116 bereit.

Die über die Exporter gelieferten Datensätze umfassen sowohl die Klassifizierung der Anomalien (Attack oder Benign) als auch sämtliche Features, die für die jeweilige Detektion herangezogen wurden. Dieser umfassende Ansatz ist notwendig, da eine zukünftige Optimierung der Gesamtdetektion mittels eines Machine-Learning-Modells vorgesehen ist. Um ein qualitativ hochwertiges Training des Modells zu ermöglichen, kann eine möglichst breite Basis an Inputdaten den Lernprozess signifikant verbessern [5].

Die Grundlage hierfür bildete eine detaillierte Analyse der Detektionseinheiten sowie ein tiefgreifendes Verständnis der bereitgestellten Datenstrukturen, wie bereits in Kapitel 3.3 (Detection Einheiten der Ladeinfrastruktur) dargelegt.

Der Threat Report importiert diese Daten über Prometheus und überführt sie in ein CSV-Format. Das CSV-Format wurde aufgrund seiner weitreichenden Kompatibilität und einfachen Handhabung und Menschenlesbarkeit gewählt. Zusätzlich überschreiten die generierten Thread Reports eine Dateigröße von maximal 30 kB nicht und somit bieten Zeilenbasierte formate wie CSV oder JSON gegenüber spaltenorientierten Formaten wie *Parquet* keine Nachteile. Im Gegenteil Parquet-spezifischer Metadaten-Overhead kann bei derart kleinen Dateien den Großteil der Dateigröße ausmachen, während die Vorteile (Kompression, spaltenbasierter Zugriff) erst bei Dateigrößen im Bereich mehrerer hundert Megabyte relevant werden [12] [2].

4.2 Zeitliche Diskretisierung und Intervalldefinition

Um die Systemressourcen des *Raspberry Pi Zero 2 W* zu schonen und gleichzeitig das Datenaufkommen zu begrenzen, ohne die Detektionssicherheit zu beeinträchtigen, wurde ein Zeitfenster von 60 Sekunden gewählt, das in 30-Sekunden-Intervallen unterteilt ist. Dieser Ansatz ermöglicht es, umfassende Informationen der Detektionseinheiten zu erhalten. Die Parameter für diese Zeitintervalle wurden bewusst variabel gestaltet. Dies schafft die Grundlage für zukünftige Untersuchungen, wie sie in Kapitel 8 (Diskussion und Forschungslücke) beschrieben sind, um den Einfluss unterschiedlicher Zeitfenster auf die Qualität der Reports empirisch zu evaluieren.
Die aktuell implementierte Sequenz umfasst drei Datenpunkte: den aktuellen Zeitpunkt (0 Sekunden) sowie die Zustände der Detektionseinheiten vor 30 und 60 Sekunden. Durch diese Struktur wird eine Timeline generiert, die den zeitlichen Verlauf der Detektion über drei Messpunkte abbildet, welches eine verbesserte Darstellung des zeitlichen Verlaufs ermöglicht als isolierte Zeitpunkte zu betrachten [17] [5].

4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse

Neben den instantanen Merkmalswerten (Features) und Anomalie-Indikatoren der Detektionseinheit werden zusätzlich historische Daten mittels *Sliding Windows* importiert. Pro Threat Report werden drei dieser Fenster definiert, welche jeweils den aktuellen Datenpunkt in Relation zu zwei vorangegangenen Zeitpunkten setzen:

Logikfrage.

ist das wirklich immer im vergleich oder zeigt mir das die min/max/avg werte innerhalb des fensters an?

- **Window 1:** 0 Sekunden im Vergleich mit -30 s und -60 s
- **Window 2:** -30 Sekunden im Vergleich mit -60 s und -90 s
- **Window 3:** -60 Sekunden im Vergleich mit -120 s und -150 s

Innerhalb dieser Zeitfenster werden für die statistische Auswertung anhand klassischer Kennzahlen berechnet [5] [1]:

attack scenario

Deutsch schreiben!

- **Mittelwert:** *Average* der Durchschnittswert innerhalb des Fensters.
- **Extremwerte:** *Minimum* und *Maximum* im jeweiligen Intervall.
- **Median:** Der Zentralwert der Datenpunkte.
- **Zuwachs (Increase):** Die relative oder absolute Veränderung innerhalb des Fensters.
- **Summe:** Die kumulierten Werte des jeweiligen Zeitraums.

Durch diese Methodik entsteht ein umfassender Report, der nicht nur eine Momentaufnahme der Detektionseinheiten liefert, sondern auch die zeitliche Entwicklung über die letzten 60 Sekunden bzw. durch die Sliding Windows bis zu 150 Sekunden in die Vergangenheit abbildet. Diese Datenbasis dient als Input für eine spätere Aufbereitung und trainings eines Machine-Learning-Modells, welcher in Kapitel 8 (Diskussion und Forschungslücke) beschrieben sind.
Der für diesen Abschnitt implementierte Quellcode, welcher die Logik der Datenabfrage sowie die Generierung des Reports umfasst, ist im Anhang unter Listing 9.1 (`reporter.py`) und Listing 9.2 (`queries.py`) dokumentiert.

5 Theoretische Grundlagen der Informationsfusion

5.1 Vergleich von Aggregationsmethoden

Rechtschreiubng Nun da jeder Knoten automatisiert einen umfangreichen Thrat Report erstellt und eine Vielzahl von Informationen über die Bedrohungslage and seine Nachbarn weitergeben kann, stellt sich die Frage, wie diese Informationen am effektivsten zur verbesserten Erkennung von Angriffen des einzelnen Knotens genutzt werden kann. Hier ist folgende problemstellung umfrangreich zu verstehen

verbessere den abschnitt auf Rechtschreibung

- Jeder Knoten erstellt einen standardisierten Threat Report wie in Kapitel (Konzeption des Threat Reporters beschrieben) 4, der Informationen über die Bedrohungslage enthält.
- Die Informationen werden an die Nachbarn in einem Peer-to-Peer Netzwerk weitergegeben über die Pub/Sub Infrastruktur wie in Kapitle (Systemarchitektur und Detection Framework) 3 beschrieben, in einem festen Intervall von 60 Sekunden.
- Für die verbesserte Erkennung jedes einzelnen Knotens stehen zu jeden intervall eine variable anzahl an Threat Reports von den Nachbarn zur Verfügung, dies ist dem dynamischen aufbau der Netzwerktopologie geschuldet.
- Es gilt eine Methode zu entwickeln, welche die relevanz der einzelnen Knoten bewertet und somit die Gewichtung der Informationen jedes einzelnen Threat Reports dynamisch und intelligent anpasst.

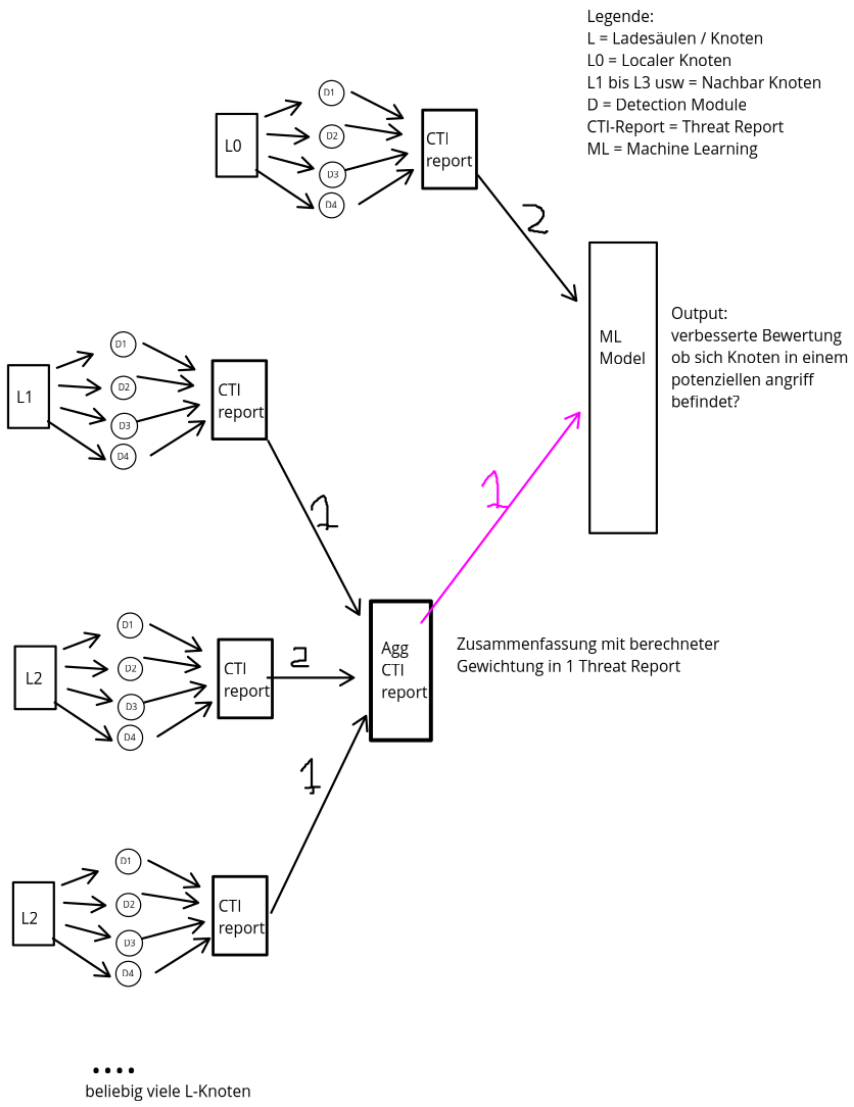

Figure 7: Aufbau der Fusion von Threat Reports aus Nachbarknoten

klassiche problem von neuronalen netzten ist, dass sie immer einen festen Eingabe Menge benötigen, folgend wird untersucht, welche methoden es gibt um dies sicherzustellen, da die zahl er erhaltenen CTI-reports variabel ist darauf aber ein Machine Learning algorithmus trainiert werden muss

baue den part noch in die erklärung mit ein.

5.1.1 Statische Aggregation

In der einschlägigen Literatur existiert eine Vielzahl von Methoden zur Aggregation von Informationen aus heterogenen Quellen. Primäre Ziele dieser Verfahren sind die Unterstützung der Entscheidungsfindung, die Vermeidung von Informationsüberlastung sowie die Identifikation von Korrelationen.

Ein technisch fundierter Ansatz besteht zunächst in der Normalisierung der Daten unterschiedlicher Quellen, um die Vergleichbarkeit zu gewährleisten. Im nächsten Schritt werden Beziehungen zwischen den Quellen und deren Inhalten identifiziert, bevor die einzelnen Meldungen (Alerts) schließlich zu übergeordneten „Situationen“ zusammengefasst werden. Eine solche Aggregation

kann beispielsweise auf Basis identischer Ursprungssysteme oder spezifischer Angriffsklassen erfolgen [3].

Bei der statischen Aggregation liegt der Fokus auf vorab manuell definierten Regeln. Diese sind regelbasiert konzipiert und verfügen über keine lernenden Komponenten. Ein typisches Szenario für diesen Ansatz ist die situationsbezogene Aggregation: Hierbei werden unterschiedliche Alerts nur dann zu einer Situation konsolidiert, wenn sie dieselbe Quelle, dasselbe Ziel und dieselbe Angriffsklasse aufweisen. Die Granularität dieser Zusammenfassung wird dabei durch statisch konfigurierte Schwellenwerte gesteuert [3]. Als gängige mathematische Aggregationsmethoden kommen in diesem Kontext nach der gruppierung die Berechnung von Minimum, Maximum, Durchschnitt oder der Standardabweichung zum Einsatz [6, 7].

5.1.2 Padding und Masking

Padding und Masking sind essenzielle Verfahren bei der Verarbeitung von Sequenzdaten, um die Inferenz mit variierenden Sequenzlängen zu ermöglichen. Diese Mechanismen sind standardmäßig in modernen Deep-Learning-Frameworks integriert, wie beispielsweise im Parameter `key_padding_mask` der `torch.nn.MultiheadAttention`-Klasse in PyTorch [10].

Der Prozess des Paddings definiert eine maximale Sequenzlänge $n_{\max}$. Sequenzen, die diese Länge unterschreiten, werden mit speziellen `[PAD]`-Token aufgefüllt, bis sie die Dimension $n_{\max}$ erreichen. Damit diese künstlich hinzugefügten Token das Training oder die Aggregation nicht verfälschen, wird eine Maskierung (Masking) eingesetzt. Diese stellt sicher, dass die entsprechenden Positionen während der Berechnung der Attention-Scores ignoriert werden.

In der wegweisenden Arbeit von Vaswani et al. (2017) [13] bildet dieses Prinzip die Grundlage für die effiziente Batch-Verarbeitung innerhalb der Transformer-Architektur. Hierbei verhindert das Masking, dass das Modell Aufmerksamkeit auf Padding-Bereiche richtet (siehe Abbildung 8).

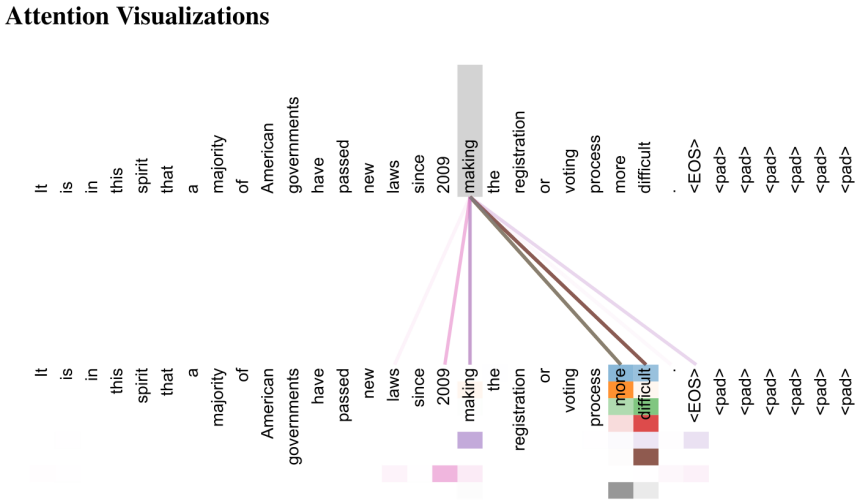

Figure 8: Visualisierung der Attention-Mechanismen (adaptiert nach Figure 3 aus [13])

Ebenso findet dieses Verfahren Anwendung in der BERT-Architektur von Devlin et al. (2019) [4]. Hierbei werden Eingabesequenzen auf eine feste Länge von meist 512 Token gepaddet, wobei

das Masking sowohl für das Padding als auch für die spezifische Trainingsmethode des *Masked Language Modeling* (MLM) verwendet wird.

5.1.3 Deep Sets

Im Gegensatz zu den in Abschnitt 5.1.2 beschriebenen Verfahren, die auf einer fixen Anzahl von Elementen basieren, verfolgen *Deep Sets* einen mengenbasierten Ansatz. Dabei wird die Eingabe als eine ungeordnete Menge X betrachtet. Das Modell lernt hierbei, die Relevanz der einzelnen Elemente autonom zu bewerten und die Gewichtung der Informationen dynamisch anzupassen.

Eine fundamentale Anforderung an die verarbeitende Funktion $f(X)$ ist die *Permutationsinvarianz*. Dies bedeutet, dass die Ausgabe der Funktion unabhängig von der Reihenfolge der Elemente innerhalb der Menge sein muss [16].

Theorem 1 (Zaheer et al., 2017)**.** *Eine Funktion $f(X)$ auf einer Menge X, deren Elemente aus einem abzählbaren Universum stammen, ist genau dann permutationsinvariant, wenn sie sich wie folgt dekomponieren lässt:*

$$f(X) = \rho\left(\sum_{x \in X} \phi(x)\right)$$

wobei ϕ und ρ geeignete Transformationen (typischerweise neuronale Netze) darstellen.

Während die Summenbildung die theoretische Grundlage für die Aggregation bildet, werden in der Praxis häufig auch der Mittelwert (*Mean*) oder das Maximum (*Max*) verwendet, da diese Operationen ebenfalls die Bedingung der Permutationsinvarianz erfüllen. Soelch et al. zeigen jedoch auf, dass die Wahl der Aggregationsfunktion einen erheblichen Einfluss auf die Modellperformance hat, und schlagen als Alternative lernbare Aggregationsmechanismen vor [11].

5.1.4 Set2Set

Obwohl auch Ansätze der vorheringen Aggregations Methoden variable Eingabelängen verarbeiten können, unterscheiden sie sich von Set2Set primär in der Art der Informationsverdichtung. Während herkömmliche Methoden die Eingabemenge meist durch eine statische Aggregationsfunktion (z. B. Mean-Pooling) in einen fixen Merkmalsraum transformieren, nutzt Set2Set einen iterativen *Read-Process*. Durch die Kombination aus Attention-Mechanismen und einem rekurrenten Zustand kann das Modell die Menge mehrfach explorieren. Dies ermöglicht es, komplexe Abhängigkeiten zwischen den Elementen aufzulösen, die bei einer einfachen Summation oder Mittelwertbildung verloren gingen [15].

Algorithmische Experimente, wie das Sortieren von Zahlenfolgen, belegen, dass Modelle eine höhere Lern- und Generalisierungsfähigkeit aufweisen, wenn sie nicht durch eine willkürliche Reihenfolge der Eingabedaten eingeschränkt werden [15]. Auch die in Abschnitt 5.1.2 behandelten Transformer-Architekturen wird mit einer Form des Set2Set-Ansatzes erweitert [13].

5.1.5 Graph Neural Networks and Pooling

Graph Neural Networks (GNNs) bieten eine leistungsstarke Methode zur Verarbeitung von Daten, die in Form von Graphen vorliegen. Knoten repräsentieren dabei Entitäten (z. B. IP-Adressen),

während Kanten die Beziehungen zwischen diesen Entitäten modellieren und die zugehörigen Flow-Informationen als Kantenmerkmale tragen [8, Sec. III-A].

Durch iteratives Aggregieren der Nachbarschaftsinformationen können GNNs sowohl topologische Muster als auch merkmalsbasierte Strukturen im Netzwerkgraphen erkennen. Das Pooling erfolgt hier, wie bereits in Abschnitt 5.1.4 beschrieben, mittels Mean-Pooling über die Kanten-Features der Nachbarschaft [8, Sec. IV-A, Eq. 6].

6 Methodik: Gewichtete Aggregation von Threat Reports

6.1 Anforderungen an die Gewichtungsfunktion

Für die Konzeption der Gewichtungsfunktion ergeben sich aus dem Anwendungskontext der CTI-Reports folgende zentrale Anforderungen:

- **Kardinalitätsinvarianz und Extraktion eines konsistenten Merkmalsraums:** Da die Anzahl der liefernden Knoten und damit der eingehenden CTI-Reports im operativen Betrieb stetig fluktuiert, muss die Gewichtungsfunktion eine beliebige Anzahl von Input-Mengen verarbeiten können. Ziel ist die Extraktion einer gleichbleibenden Informationsmenge (Fixed-Size Representation) aus diesen variablen Eingangsdaten [16, Sec. 1] [14, Sec. 1]. Dies stellt sicher, dass der resultierende Merkmalsvektor unabhängig von der ursprünglichen Report-Menge eine uniforme Struktur aufweist, was die notwendige Voraussetzung für die Performanz und Stabilität nachgelagerter Machine-Learning-Algorithmen ist (siehe Kapitel 8).
- **Relevanzbasierte Gewichtung der Detektionen:** Um die Qualität der aggregierten Ergebnisse zu steigern, muss die Funktion die Relevanz der einzelnen CTI-Reports differenziert bewerten. Dies erfolgt auf Basis der in den Reports enthaltenen Einzeldetektionen. Durch die Einbeziehung dieser qualitativen Informationen in den Aggregationsprozess wird eine präzisere und aussagekräftigere Informationsverdichtung ermöglicht, wobei CTI-Reports mit einer erhöhten Anzahl an Einzeldetektionen eine höhere Relevanz in der Aggregation zugeordnet wird.
- **Asymptotisches Sättigungsverhalten der Gewichtung:** Um eine übermäßige Dominanz einzelner Berichte mit extrem hohen Detektionszahlen zu vermeiden, darf die Gewichtungsfunktion kein unbegrenztes Wachstum aufweisen. Stattdessen wird eine Funktion gefordert, die gegen einen fest definierten Grenzwert (Asymptote) konvergiert. Durch dieses Sättigungsverhalten wird sichergestellt, dass zusätzliche Detektionen ab einem gewissen Schwellenwert nur noch einen marginalen Einfluss auf die Gesamtgewichtung haben, was eine Verzerrung der Ergebnisse durch statistische Ausreißer verhindert.

6.2 Auswahl der Gewichtungsfunktion

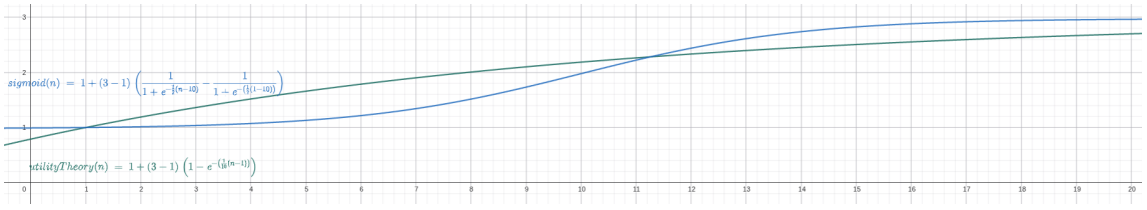

Figure 9: Vergleich der Sigmoid-Funktion und der Utility-Theory-Funktion. Die X-Achse zeigt die Anzahl positiver Detektionen abzüglich des Initialwertes ($n-1$), die Y-Achse den resultierenden Gewichtungsfaktor.

6.2.1 Utility-Theory-Funktion

Asymptotische Gewichtungsfunktion: Zur Modellierung der Relevanz wird eine Funktion gewählt, die dem Prinzip des abnehmenden Grenznutzens aus der Entscheidungstheorie folgt. Hierzu wird die *Negative Exponential Utility Function* herangezogen, welche durch eine konstante absolute Risikoaversion (CARA) charakterisiert ist [9, Sec. 8].

Durch eine positive affine Transformation [9, Sec. 3] wird die Funktion so skaliert, dass sie monoton steigend gegen einen definierten Grenzwert W_{max} konvergiert, wobei zusätzliche Detektionen einen abnehmenden Grenzinformationsgewinn (*diminishing returns*) darstellen [9, Sec. 1]. Dies verhindert eine Verzerrung durch CTI-Reports mit extrem vielen positiven Einzeldetektionen und stellt eine robuste Aggregation sicher.

Die Gewichtung der Reports erfolgt über die folgende Funktion:

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot (1 - e^{-K \cdot n}) \tag{1}$$

Wie die Kurve verdeutlicht, erfahren CTI-Reports mit einer steigenden Anzahl positiver Detektionen (n) eine kontinuierlich höhere Gewichtung (w). Diese konvergiert gegen einen definierten Grenzwert von $W_{\text{max}} = 3$, wodurch eine übermäßige Dominanz einzelner Berichte mit extrem hohen positiven Einzeldetektionen (Outlier) unterdrückt wird [9, Sec. 8]. Gleichzeitig stellt der Achsenabschnitt bei $n = 1$ sicher, dass jeder Bericht mit einem Basisgewicht von $W_{\text{base}} = 1$ in die Aggregation einfließt, um eine Mindestrelevanz aller vorliegenden Informationen zu gewährleisten.

Die praktische Umsetzung der beschriebenen Gewichtungslogik ist in Listing 9.3 (`aggregation.py`) innerhalb der Funktion `add_report_weight` dokumentiert.

6.2.2 Sigmoid-Funktion

Asymptotische Gewichtungsfunktion mit S-förmigem Verlauf: Als Alternative zur Utility-Theory-Funktion wurde die *Logistische Sigmoid-Funktion* evaluiert. Sie ist im Bereich des maschinellen Lernens weit verbreitet und findet insbesondere in neuronalen Netzen Anwendung, etwa als Aktivierungsfunktion bei der Klassifikation sowie in Graph Attention Networks zur Aggregation von Nachbarknoten [14, Sec. 2.1]. Ein weiterer Vorteil des Attention based ansatztes ist das dieser eine beliebige Anzahl von Inputs verarbeiten kann [14, Sec. 1].

Durch eine positive affine Transformation sowie eine Verschiebung um den Ankerpunkt wird die Funktion so skaliert und normiert, dass bei $n = 1$ stets $w(1) = W_{\text{base}}$ gilt. Dies stellt sicher, dass jeder CTI-Report mindestens mit dem Faktor W_{base} in die gewichtete Aggregation einfließt. Im Code wird $n = 1$ als Standardwert verwendet, sodass Reports ohne positive Detektionen ($n_{\text{attacks}} = 0$) mit der gewichtung = 1 bewertet werden.

Die Gewichtung der Reports erfolgt über die folgende Funktion:

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot \left(\frac{1}{1 + e^{-k(n - n_0)}} - \frac{1}{1 + e^{-k(1 - n_0)}} \right) \tag{2}$$

Der Subtraktionsterm $\frac{1}{1+e^{-k(1-n_0)}}$ stellt den Ankerpunkt bei $n = 1$ sicher: Er verschiebt die gesamte Kurve vertikal so, dass $w(1) = W_{\text{base}}$ exakt erfüllt ist, unabhängig von der Wahl der Parameter k und n_0.

Im Gegensatz zur Utility-Theory-Funktion, die rein konkav verläuft und den stärksten Anstieg bei $n = 1$ aufweist, besitzt die Sigmoid-Funktion durch den Wendepunkt bei n_0 einen S-förmigen Verlauf [14, Sec. 2.1]. Für kleine $n < n_0$ wächst das Gewicht zunächst moderat, steigt dann steil an und flacht für $n \gg n_0$ asymptotisch gegen $W_{\max}$ ab. Dies modelliert die Annahme, dass eine Mindestzahl an Detektionen überschritten werden muss, bevor ein Report als signifikant eingestuft wird.

Wie in Abbildung 9 zu erkennen ist, unterscheiden sich beide Funktionen vor allem im Bereich kleiner Detektionszahlen: Während die Utility-Theory-Funktion Reports mit wenigen Detektionen bereits stark gewichtet, verzögert die Sigmoid-Funktion diesen Anstieg bis zum Wendepunkt n_0. Für große n konvergieren beide Funktionen gegen $W_{\max} = 3$, wodurch die Outlier-Dämpfung in beiden Fällen strukturell garantiert ist [9, Sec. 8].

Die Wahl zwischen beiden Funktionen wird im Rahmen von zukünftigen Arbeiten durch empirische Evaluierung der Aggregationsergebnisse auf realen CTI-Report-Datensätzen getroffen (siehe Kapitel 8).

6.3 Implementierung des Attention-basierten Selektionsmodells

Die im Rahmen dieser Arbeit entwickelten Gewichtungsfunktionen wurden in Python implementiert. Ein wesentliches Merkmal dieser Implementierung ist ihre hohe rechnerische Effizienz: Da die Gewichtung unabhängig für jeden einzelnen CTI-Report berechnet werden kann, ist der Prozess über alle Berichte hinweg vollständig parallelisierbar. Im Gegensatz zu frühen spektralen Ansätzen in der Graph-Verarbeitung sind keine rechenintensiven Matrixoperationen wie Inversionen oder Eigenwertzerlegungen erforderlich [14, Sec. 1].

Die gewählten Parameter, wie beispielsweise der Steigungsfaktor k oder der Wendepunkt n_0, dienen in der aktuellen Implementierung als Referenzwerte. Eine finale Optimierung dieser Hyperparameter erfolgt in zukünftigen Arbeiten durch eine empirische Evaluierung der Aggregationsergebnisse auf Basis realer CTI-Datensätze (siehe Kapitel 8).

- **Kardinalitätsinvarianz und Merkmalsraumextraktion** Die Anforderung der Invarianz gegenüber variablen Input-Kardinalitäten wird durch die Funktion `aggregate_weighted_reports` realisiert. In Anlehnung an den Attention-Mechanismus ermöglicht dieser Ansatz die Verarbeitung einer beliebigen Anzahl von Eingangsgrößen, während gleichzeitig ein Merkmalsvektor mit invarianter Dimension (Fixed-Size Representation) extrahiert wird [14]. Dies stellt die notwendige Kompatibilität für nachgelagerte Machine-Learning-Modelle sicher.

- **Relevanzbasierte Gewichtung** Die qualitative Differenzierung der Berichte erfolgt in der Funktion `count_attack_ongoing_per_report`. Diese extrahiert die Anzahl der Einzeldetektionen innerhalb eines CTI-Reports und überführt diese als quantitative Basis in den Gewichtungsprozess. Damit wird sichergestellt, dass die Aggregation die Informationsdichte der einzelnen Quellen berücksichtigt.

- **Asymptotisches Sättigungsverhalten** Um eine Verzerrung der Ergebnisse durch statistische Ausreißer oder extrem hohe Detektionszahlen zu verhindern, wurde ein asymptotisches Sättigungsverhalten implementiert. Die Funktionen `add_report_weight` und `add_report_weight_sigmoid` (Listing 9.3) sind so konzipiert, dass sie gegen einen definierten Grenzwert konvergieren (siehe Kapitel 6.2). Während die Utility-Theory-Funktion einen rein konkaven Verlauf aufweist, nutzt die Sigmoid-Funktion eine nicht-lineare S-Kurve zur Stabilisierung der Gewichte (siehe Abbilundg 9), ähnlich der in graph attention networks (GATs) verwendeten Aktivierungsfunktionen [14]. Eine vergleichende Evaluation beider mathematischen Modelle hinsichtlich ihrer Performance im operativen Betrieb ist Bestandteil zukünftiger Forschung (siehe Kapitel 8).

7 Evaluation

7.1 Generierung der Threat Reports

Um das System umfassend evaluieren zu können, wurden beispielhafte Threat Reports generiert, die ein breites Spektrum potenzieller Szenarien abdecken. Wie in Kapitel 3.3 (Detection-Einheiten der Ladeinfrastruktur) dargelegt, wird ein Großteil der Flooding-Attacken durch die implementierten Detektionsmechanismen erkannt. Für die Testgenerierung wurde daher eine UDP-Flooding-Attacke mittels des Tools `hping3` durchgeführt:
`hping3 -flood -udp -p 80 <IP-Adresse der Wallbox>`.
Dabei reagierte wie erwartet lediglich die *Power Temperature Detection* nicht, da diese ausschließlich physikalische Parameter wie Temperatur und Leistung überwacht und keine netzwerkspezifischen Daten analysiert. In diesem Szenario resultieren daraus fünf CTI-Reports, die variierende Feature-Werte und zeitlich leicht verschobene positive Detektionen aufweisen.

	key	original_name	n_attacks	weight
0	report_00002	threat_report_attack.csv	9	2.186861
1	report_00003	threat_report_attack4_aka_benign.csv	6	1.902377
2	report_00004	threat_report_attack5_aka_benign.csv	6	1.902377
3	report_00005	threat_report_attack_3.csv	7	2.006829
4	report_00006	threat_report_attack_2.csv	8	2.101342

Figure 10: Output der Aggregation der fünf CTI-Reports aus Szenario 1

Zur besseren Evaluation der Code-Struktur werden die relevantesten Parameter bei jeder Ausführung protokolliert und ausgegeben.

In der folgenden Abbildungen werden die kumulierten Werte des Threat Reports verdeutlicht, darunter statistische Kennzahlen wie Durchschnitt (Avg), Minimum, Maximum, Median, Anstieg (Increase) sowie die Gesamtsumme. Die Analyse erfolgt über Sliding Windows, die vom aktuellen Zeitpunkt (Offset 0s) bis zu einem Rückblick von -60s reichen. Im vorliegenden Fall ist ersichtlich, dass die *Attack Probability* der Hardware-Detection mit einer Wahrscheinlichkeit von über 0,98 identifiziert wurde. Diese sinkt im aktuellsten Zeitfenster (Offset 0s) signifikant auf einen Wert unter 0,01 ab. Dies deutet darauf hin, dass das Angriffsereignis entweder beendet oder erfolgreich abgewehrt wurde.

metric_name	instance	offset	start_time	end_time	num_samples	latest_value	avg	min	max	median	increase	sum
ids_power_bus_voltage	ids_power:2113	0s	2026-03-23T16:07:20	2026-03-23T16:08:20	3	4.853	4.878333	4.853	4.901	4.881	0.02	14.635
ids_power_bus_voltage	ids_power:2113	-30s	2026-03-23T16:06:50	2026-03-23T16:07:50	3	4.901	4.907667	4.881	4.941	4.901	0.02	14.723
ids_power_bus_voltage	ids_power:2113	-60s	2026-03-23T16:06:20	2026-03-23T16:07:20	3	4.881	4.914333	4.881	4.941	4.921	0.02	14.743
ids_hardware_attack_probability	hardware_events:2116	0s	2026-03-23T16:07:20	2026-03-23T16:08:20	3	0.005355	0.661544	0.00536	0.9897	0.989553	0.00017	1.984632
ids_hardware_attack_probability	hardware_events:2116	-30s	2026-03-23T16:06:50	2026-03-23T16:07:50	3	0.989723	0.988976	0.98765	0.9897	0.989553	0.002072	2.966928
ids_hardware_attack_probability	hardware_events:2116	-60s	2026-03-23T16:06:20	2026-03-23T16:07:20	3	0.989553	0.822489	0.49026	0.9896	0.987651	0.499289	2.467468

Figure 11: Übersicht der generierten Threat-Report-Szenarien

- **Szenario 1 – NaN-Werte (Error Handling):** Zur Untersuchung des Umgangs mit NaN-Werten (Not-a-Number). In diesem Szenario muss sichergestellt werden, dass die Generierung von Threat Reports auch bei unvollständigen oder fehlerhaften Daten robust bleibt. So kann es im laufenden Betrieb vorkommen, dass beispielsweise aufgrund von Netzwerkproblemen oder Sensor ausfällen von exportern keine Daten mehr über Prometheus bezogen werden könnnen (siehe Details zur Systemarchitektur siehe Kapitel 3 3). Die entscheidung des kompletten ausschlusses solcher Reports basiert auf der Tatsache, dass

ein Report ohne valide Daten keine Informationsgrundlage bietet, um den Angriffsstatus der Wallbox zu beurteilen. Eine Substitution der NaN-Werte durch 0 oder andere Default-Werte wurde als nicht sinnvoll erachtet, da dies das Ergebnis der Bedrohungsanalyse verfälschen könnte Somit wäre eine Ersetzung von NaN-Werten durch 0 oder andere Werte nicht sinnvoll gewesen..

Quelle dafür hinzufügen

So wurde im code eine funktion implementiert welche die generierten CTI-Reports auf NaN-Werte überprüft und diese gegebenenfalls ausschließt siehe Listing 9.4 unter der funktion `filter_reports_with_NaN`. Aufgrund dieser implementierung wird erwartet, dass der erstellte CTI-Report mit ausgefallenen exporter und daraus resolutierenden NaN Werten bereits vor der Aggregation mit anderen Reports ausgeschlossen wird, und der User über diesen ausschluss informiert wird.

Um dieses Szeanario zu erzeugen, wurde während der kontinuierlichen Reporterstellung ein Datenexporter terminiert und neugestartet. Dies führte zu einer erhöhten Anzahl an NaN-Werten in den generierten CTI-Reports, da Feature-Werte über Prometheus nicht mehr bezogen werden konnten (Details zur Systemarchitektur siehe Kapitel 3).

metric_name	instance	offset	start_time	end_time	nur	latest_va	avg	min	max	median	increase	sum
ids_hardware_feature_bus_access	hardware_events:2116	0s	2026-03-23T16:07:04.0	2026-03-23T16:08:04	3	95311	86768	70130	95311	94863	25181	260304
ids_hardware_feature_bus_access	hardware_events:2116	-30s	2026-03-23T16:06:34.0	2026-03-23T16:07:34	3	94863	nan	nan	nan	70130	24733	nan
ids_hardware_feature_bus_access	hardware_events:2116	-60s	2026-03-23T16:06:04.0	2026-03-23T16:07:04	3	70130	nan	nan	nan	nan	0	nan

Figure 12: Auszug eines CTI-Reports aus Szenario 2 mit NaN-Werten

Infolgedessen wurde der betroffene CTI-Report bereits vor der Aggregation mit anderen Reports ausgeschlossen.

```
gebung/DataAggregator$ python3 -m cti_agg.pipeline

/home/yannic/Desktop/Master_rbg/Semester1/Forschungsarbeit1/code/EVSE_Testumgebung/DataAg
gregator/cti_agg/pipeline.py:18: UserWarning: report_00001 (threat_report_nan_value.csv)
has NaN value(s) and got ignored
  manifest = filter_reports_with_NaN(manifest, h5_path=h5_path)
           key                          original_name  n_attacks    weight
0  report_00002              threat_report_attack.csv          9  2.186861
1  report_00003  threat_report_attack4_aka_benign.csv          6  1.902377
2  report_00004  threat_report_attack5_aka_benign.csv          6  1.902377
3  report_00005            threat_report_attack_3.csv          7  2.006829
4  report_00006            threat_report_attack_2.csv          8  2.101342
```

Figure 13: Auszug der versuchten Aggregation aus Szenario 2 mit NaN-Werten

Aus diesem Auszug der Aggregation von Reports Kann versgestellt werden das das zu erwartetende ergebniss eingetroffen ist.

- **Szenario 2 – Verschobene timeline** Wie wurde die Timeline manipuliert Auswirkung auf validate_same_metrics Aggregation
- **Szenario 3 – Alle Detections negativ** Baseline-Report (Normalzustand) Dient als Referenz für Kapitel 7.2
- **Szenario 4 – Manuell manipulierter Ausreißer-Report** Alle Detections auf positiv gesetzt Dient als Grundlage für Ausreißer-Analyse in Kapitel 7.2

7.2 Vergleich der Gewichtungsfunktionen

Vergleich der Gewichtungsfunktion: erstmal die baisc durch einfachs testen des codes welche reports hat er rausgefiltert? NaN und verschobene Timeline? Das alles kurz erklären das er die reports richtig rausgefiltert hat. Dann die gewichtung der einzelenen reports anschauen und wie sehr der ausreiser alles verändert hat oder eben nicht, wie ist die veränderung des aggregierten Reports anhand der anzahl der inkludierten Threat reports nochmal den unterschied der S kurzve von Sigmoid und im vergleich die UtilityTheory funktion welche am anfang einen starken anstieg hat, wie könnte sich das in der theory auswikrungen auf den späteren einsatz der jeweiligen formel haben. Utitlity Theory wallboxen welche evtl. nur eine detection kurzeitig positiv einzeldetection liefert wird im vergleich zur sigmoid funktion direkt mehr bewertet und einbezogen. Andrerseit könnte es bei der sigmoid funktion passieren das ein report welcher nur eine detection bei einem angriff auslösst und eigeneltich richtig ist nur minimal höhrere gewichtung als andere bekommt.

Gewicht jedes Reports unter Utility-Theory vs. Sigmoid tabellarisch gegenüberstellen

Fokus: Wie stark verändert der Ausreißer-Report (Szenario 5) den aggregierten Output?

Metriken: MAE / relative Abweichung vom Aggregat ohne Ausreißer als Baseline Schrittweise Reports hinzufügen (1 -> N) und Veränderung des aggregierten Feature-Vektors beobachten

Zeigt: Wie stabil konvergiert der Aggregat mit steigender Knotenzahl?

Utility Theory: starker Anstieg bei kleinen n -> Reports mit nur einer positiven Einzeldetection werden direkt stärker gewichtet

Sigmoid: moderater Anstieg bis Wendepunkt n0 -> Reports mit wenigen Detections erhalten kaum erhöhte Gewichtung

Implikation für den Einsatz:

Utility Theory: sensitiver gegenüber schwachen Signalen, höheres Risiko False-Positive-Verstärkung

Sigmoid: konservativer, benötigt Mindestzahl an Detections für signifikante Gewichtungserhöhung, möglicherweise blinde Flecken bei isolierten Einzeldetections

7.3 Echo Chamber Analyse

Echo chamber effekt effekt erstmal anhand von literatur erklären mathematisch zeigen, wie sich der effekt auswirkt einmal bei der utitly theory fomel und einmal bei der sigmoid formel und unterschied kurz erklären was evtl. besser geeigent wäre

Echo-Chamber-Effekt im Kontext kollaborativer IDS-Systeme definieren

Relevante Literatur (z.B. Debar und Wespi , Heigl et al.)

Übertragung auf das vorliegende P2P-Szenario: Mehrheit fehlerhafter Knoten beeinflusst Aggregat Szenario formalisieren: M Knoten melden fälschlicherweise Attack, K Knoten korrekt Benign

Zeige anhand der Aggregationsformel aus Kapitel 6.3, wie das Gewichtsverhältnis M/K den Aggregat-Wert verzerrt Berechnung des aggregierten Outputs für das obige Szenario mit Formel (1)

Durch den frühen starken Anstieg: bereits wenige fehlerhafte Knoten mit mittlerer Detektionszahl kippen den Aggregat Effekt unter Sigmoid-Funktion:

Selbes Szenario mit Formel (2)

Durch Wendepunkt n0: fehlerhafte Knoten mit wenigen Detections haben geringeren Einfluss, starke Ausreißer jedoch ähnlich wie Utility Theory

Tabellarischer Vergleich: Anfälligkeit beider Funktionen gegenüber Echo-Chamber unter verschiedenen M/K-Verhältnissen

Aussage: Keine der Funktionen löst das Problem vollständig -> Verweis auf Kapitel 8.3 als offene Forschungsfrage

Mögliche Gegenmaßnahme als Ausblick: reputationsbasierte Gewichtung, Byzantine-fault-tolerante Aggregation

8 Diskussion und Forschungslücken

8.1 Auswirkungen unterschiedlicher Gewichtungsstrategien auf die Report-Qualität

8.2 Potenzial der LLM-gestützten Aufbereitung für externe CTI-Nutzung

8.3 Untersuchung des Einflusses von Report-Aggregation auf Detectionsergebnisse (Echo-Chamber-Effekt?)

- weniger Daten bessere ergebnisse wie in kapitel Konzetiop des Threat Reports beschrieben? - 30 sekunden intervall gut oder soll es verößert verkleinert werden?

9 Quellcode

Listing 9.1: reporter.py – Logik der Report-Generierung

```
import time
import requests
import csv
import statistics
from datetime import datetime, timedelta
from queries import get_queries  # liefert NUR Basis-PromQL-Expressions
    zurück

# === KONFIGURATION ===
PROMETHEUS_URL = "http://192.168.0.25"  # Prometheus-Endpunkt
IPFS_API_URL = "http://localhost:3001"  # IPFS API-Endpunkt
OUTPUT_CSV = "threat_report.csv"

# Zeitraum
TIMESPAN = 1  # Minuten
END_TIME = datetime.utcnow()
START_TIME = END_TIME - timedelta(minutes=TIMESPAN)
STEP = "30s"

# Zusätzliche Zeitversätze (in Sekunden)
OFFSETS = [0, -30, -60]


def query_prometheus_range(query, start, end, step):
    """Fragt Prometheus nach einem Zeitbereich (range query)."""
    url = f"{PROMETHEUS_URL}/api/v1/query_range"
    params = {
        "query": query,
        "start": start.isoformat() + "Z",
        "end": end.isoformat() + "Z",
        "step": step,
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    result = response.json()

    if result["status"] != "success":
        raise RuntimeError(f"Fehler␣bei␣Abfrage:␣{result}")

    return result["data"]["result"]


def compute_basic_stats(values):
    """Berechne avg, min, max, median aus einer Liste von Floats."""
    if not values:
        return None
    try:
        avg = sum(values) / len(values)
        mn = min(values)
```

```
        mx = max(values)
        med = statistics.median(values)
    except Exception:
        return None

    return {"avg": avg, "min": mn, "max": mx, "median": med}


def compute_increase(values):
    """Berechnet eine einfache 'increase' Approximation aus einer Liste
        von Floats."""
    if len(values) < 2:
        return None
    inc = 0.0
    prev = values[0]
    for v in values[1:]:
        delta = v - prev
        if delta > 0:
            inc += delta
        prev = v
    return inc


def main():
    QUERIES = get_queries(START_TIME, END_TIME)

    with open(OUTPUT_CSV, mode="w", newline="", encoding="utf-8") as file
        :
        writer = csv.writer(file)
        # Kopfzeile
        writer.writerow([
            "metric_name",
            "instance",
            "offset",
            "start_time",
            "end_time",
            "num_samples",
            "latest_value",
            "avg",
            "min",
            "max",
            "median",
            "increase",
            "sum"
        ])

        for metric_name, promql in QUERIES.items():
            print(f"Metrik: {metric_name} | Query: {promql}")
            for offset in OFFSETS:
                offset_end = END_TIME + timedelta(seconds=offset)
                offset_start = offset_end - timedelta(minutes=TIMESPAN)
```

```
                series_list = query_prometheus_range(promql, offset_start
                    , offset_end, STEP)

                for series in series_list:
                    metric_labels = series.get("metric", {})
                    instance = metric_labels.get("instance", "unknown")

                    raw_values = []
                    for ts, value_str in series.get("values", []):
                        try:
                            v = float(value_str)
                        except ValueError:
                            continue
                        raw_values.append(v)

                    if not raw_values:
                        continue

                    stats = compute_basic_stats(raw_values)
                    inc = compute_increase(raw_values)
                    total_sum = sum(raw_values)
                    latest_value = raw_values[-1] if raw_values else ""

                    if stats is None:
                        continue

                    writer.writerow([
                        metric_name,
                        instance,
                        f"{offset}s",
                        offset_start.isoformat() + "Z",
                        offset_end.isoformat() + "Z",
                        len(raw_values),
                        f"{latest_value:.6f}",
                        f"{stats['avg']:.6f}",
                        f"{stats['min']:.6f}",
                        f"{stats['max']:.6f}",
                        f"{stats['median']:.6f}",
                        f"{inc:.6f}" if inc is not None else "",
                        f"{total_sum:.6f}",
                    ])

    print(f"CSV-Datei␣erstellt:␣{OUTPUT_CSV}")


if __name__ == "__main__":
    main()
    #time.sleep(TIMESPAN)  # Der Report wird nur einmal pro Minute
        aktualisiert
    #print("Warte auf nächste Aktualisierung...")
    #
    # Hier testweise eine testdatei veröffentlichen
    cid = publish_csv("./testfile.csv", IPFS_API_URL)
```

Listing 9.2: queries.py – Logik der Datenbankabfragen

```
from typing import Dict
from datetime import datetime

def get_queries(start_time: datetime, end_time: datetime) -> Dict[str,
    str]:
    """Gibt NUR Basis-PromQL-Metriken zurück (KEINE Aggregationen)."""
    return {
        # System Status
        "attack_ongoing": "attack_ongoing",

        # Power Metrics (Basis)
        "ids_power_bus_voltage": "ids_power_bus_voltage_V",
        "ids_power_current": "ids_power_current_mA",
        "ids_power_power": "ids_power_power_mW",
        "ids_power_shunt_voltage": "ids_power_shunt_voltage_mV",
        "ids_power_lstm_mse": "ids_power_lstm_mse",
        "ids_power_isolation_score": "ids_power_isolation_score",

        # Temperature Metrics (Basis)
        "ids_temperature": "ids_temperature",
        "evse_thermo_current_temp": "evse_thermo_current_temp_celsius",

        # Wattmeter Status
        "ids_wattmeter": "ids_wattmeter",

        # Anomaly Detection
        "ids_power_isolation_forest_anomaly": "
            ids_power_isolation_forest_anomaly",
        "ids_power_lstm_anomaly": "ids_power_lstm_anomaly",

        # Transformer ML Predictions (Basis)
        "transformer_prediction": "transformer_prediction",
        "ids_transformer_attack_probability": "
            ids_transformer_attack_probability",
        "ids_transformer_shunt_voltage": "ids_transformer_shunt_voltage",
        "ids_transformer_bus_voltage": "ids_transformer_bus_voltage",
        "ids_transformer_current": "ids_transformer_current",
        "ids_transformer_power": "ids_transformer_power",
        "ids_transformer_raw_prediction": "ids_transformer_raw_prediction
            ",

        # xLSTM Metrics
        "ids_xlstm_prediction": "ids_xlstm_prediction",
        "ids_xlstm_attack_probability": "ids_xlstm_attack_probability",
        "ids_xlstm_reconstruction_loss": "ids_xlstm_reconstruction_loss",

        # Ensemble
        "ids_ensemble_final_decision": "ids_ensemble_final_decision",

        # Hardware Features (Basis)
        "ids_hardware_feature_node_stores": "
```

```
            ids_hardware_feature_node_stores",
        "ids_hardware_feature_bus_access": "
            ids_hardware_feature_bus_access",
        "ids_hardware_feature_skb_kfree_skb": "
            ids_hardware_feature_skb_kfree_skb",
        "ids_hardware_feature_cpu_cycles": "
            ids_hardware_feature_cpu_cycles",
        "ids_hardware_feature_l2d_cache_wb_victim": "
            ids_hardware_feature_l2d_cache_wb_victim",
        "ids_hardware_feature_syscalls_sys_enter_fcntl": "
            ids_hardware_feature_syscalls_sys_enter_fcntl",
        "ids_hardware_feature_sched_sched_stat_runtime": "
            ids_hardware_feature_sched_sched_stat_runtime",
        "ids_hardware_feature_st_spec": "ids_hardware_feature_st_spec",
        "ids_hardware_feature_syscalls_sys_enter_kill": "
            ids_hardware_feature_syscalls_sys_enter_kill",
        "ids_hardware_feature_fib_fib_table_lookup": "
            ids_hardware_feature_fib_fib_table_lookup",
        "ids_hardware_feature_sock_inet_sock_set_state": "
            ids_hardware_feature_sock_inet_sock_set_state",
        "ids_hardware_feature_l2d_cache_wb": "
            ids_hardware_feature_l2d_cache_wb",
        "ids_hardware_feature_unaligned_st_spec": "
            ids_hardware_feature_unaligned_st_spec",
        "ids_hardware_feature_mem_access_rd": "
            ids_hardware_feature_mem_access_rd",
        "ids_hardware_feature_l2d_cache_refill_wr": "
            ids_hardware_feature_l2d_cache_refill_wr",
        "ids_hardware_feature_mmc_mmc_request_start": "
            ids_hardware_feature_mmc_mmc_request_start",
        "ids_hardware_feature_br_mis_pred": "
            ids_hardware_feature_br_mis_pred",

        # Hardware Attack Detection
        "ids_hardware_attack_probability": "
            ids_hardware_attack_probability",
        "ids_hardware_benign_probability": "
            ids_hardware_benign_probability",
        "ids_hardware_attack_prediction": "ids_hardware_attack_prediction
            ",

        # IoT Platform Metrics (neu hinzugefügt aus neuen Metriken)
        "iotplatform_meter_bus_voltage": "iotplatform_meter_bus_voltage_V
            ",
        "iotplatform_meter_current": "iotplatform_meter_current_mA",
        "iotplatform_meter_power": "iotplatform_meter_power_mW",
        "iotplatform_meter_shunt_voltage": "
            iotplatform_meter_shunt_voltage_mV",
    }
```

Listing 9.3: aggregation.py – Implementierung der Gewichtungsfunktion und Aggregationslogik

```
from __future__ import annotations
```

```python
from pathlib import Path
import pandas as pd
import numpy as np

from .config import BASE_WEIGHT, MAX_WEIGHT, K, BINARY_INDICATORS,
    BINARY_COLS, BINARY_THRESHOLD


def add_report_weight(
    df_counts: pd.DataFrame,
    n_col: str = "n_attacks",
    out_col: str = "weight",
) -> pd.DataFrame:
    """
    Berechnet Gewichte basierend auf angepassten UtilityTheory-Funktion:
    utilityTheory(n)=W_{base}+(W_{max}-W_{base}) (1-exp^(-(K (n-1))))
    """
    out = df_counts.copy()
    n = pd.to_numeric(out[n_col], errors="coerce").fillna(0).to_numpy()
    out[out_col] = BASE_WEIGHT + (MAX_WEIGHT - BASE_WEIGHT) * (1.0 - np.
        exp(-K * n))
    return out

def add_report_weight_sigmoid(
    df_counts: pd.DataFrame,
    n_col: str = "n_attacks",
    out_col: str = "weight",
    n0: float = 5.0,  # Wendepunkt der Sigmoid-Kurve
) -> pd.DataFrame:
    """
    Berechnet Gewichte basierend auf angepassten Sigmoid-Funktion:
    sigmoid(n) = W_base + (W_max - W_base) * ( (1/(1+exp(-k(n-n0)))) -
        (1/(1+exp(-k(1-n0)))) )
    """
    out = df_counts.copy()
    n = pd.to_numeric(out[n_col], errors="coerce").fillna(0).to_numpy()

    sig_n = 1.0 / (1.0 + np.exp(-K * (n - n0)))

    # Korrektur-Term, damit die Kurve bei n=1 genau bei 0 startet
    sig_1 = 1.0 / (1.0 + np.exp(-K * (1.0 - n0)))

    # Skalierung auf den Bereich zwischen BASE_WEIGHT und MAX_WEIGHT
    out[out_col] = BASE_WEIGHT + (MAX_WEIGHT - BASE_WEIGHT) * (sig_n -
        sig_1)

    # Falls n < 1 vorkommt negative Gewichte unter BASE_WEIGHT vermeiden
    out[out_col] = out[out_col].clip(lower=BASE_WEIGHT)

    return out

def aggregate_weighted_reports(
```

```
    weights_df: pd.DataFrame,
    h5_path: str | Path = "reports.h5",
    key_col: str = "key",
    weight_col: str = "weight",
    cols: tuple[str, ...] = ("latest_value", "avg", "min", "max", "median
        ", "increase", "sum"),
) -> pd.DataFrame:
    """
    Aggregiert report_00001, report_00002, ... zu einem neuen Report:
    für jede Zelle der `cols`:
      agg = (Σ report_i * weight_i) / (Σ weight_i)

    Alle Reports müssen gleiche Zeilenanzahl haben und gleiche Index/
        Zeilenreihenfolge).
    """
    h5_path = Path(h5_path)

    num = None  # Numerator: Σ(df * w)
    denom = 0.0

    validate_same_metrics(weights_df, h5_path=h5_path, key_col=key_col)

    for _, r in weights_df.iterrows():
        key = r[key_col]
        w = float(r[weight_col])

        df = pd.read_hdf(h5_path, key=key)  # lädt DataFrame unter dem
            Key.
        x = df.loc[:, list(cols)].apply(pd.to_numeric, errors="coerce")

        if num is None:
            num = x * w
        else:
            # addiert elementweise; Align passiert über Index/Columns (
                muss matchen).
            num = num.add(x * w, fill_value=0)

        denom += w

    if num is None:
        raise ValueError("weights_df␣ist␣leer,␣nichts␣zu␣aggregieren.")
    if denom == 0:
        raise ValueError("Summe␣der␣Weights␣ist␣0,␣Division␣nicht␣möglich
            .")

    agg = num / denom
    return agg

def validate_same_metrics(
    weights_df: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> None:
```

```
    '''
    Prüft das nur aggregiert wird wenn alle Reports die gleichen
        metric_name Zeilen haben (gleicher Index, gleiche Reihenfolge).
    '''
    h5_path = Path(h5_path)

    ref_index = None
    ref_key = None

    for _, r in weights_df.iterrows():
        key = r[key_col]
        df = pd.read_hdf(h5_path, key=key)

        if ref_index is None:
            ref_index = df.index
            ref_key = key
            continue

        if not ref_index.equals(df.index):
            raise ValueError(
                f"Metric␣mismatch:␣{key}␣has␣different␣metric_name/Order␣
                    as␣{ref_key}."
            )

def binarize_indicators(aggregated_values: pd.DataFrame) -> pd.DataFrame:
    '''
    Für die in BINARY_INDICATORS gelisteten Zeilen (Index) werden die
        Werte in BINARY_COLS anhand von BINARY_THRESHOLD binarisiert:
    Wenn Wert >= BINARY_THRESHOLD folgt 1, sonst 0.
    Damit Attack Indicators keine Komma Werte haben.
    '''
    out = aggregated_values.copy()

    mask = out.index.astype(str).isin(BINARY_INDICATORS)
    if not mask.any():
        return out

    cols = [c for c in BINARY_COLS if c in out.columns]
    out.loc[mask, cols] = (out.loc[mask, cols] >= BINARY_THRESHOLD).
        astype("int64")
    return out
```

Listing 9.4: filters.py – Logik der Filterung von CTI-Reports

```
from __future__ import annotations

from pathlib import Path
import pandas as pd
import warnings

from .config import TIME_COL, MAX_TIME_SKEW_SECONDS
```

```python
def filter_reports_with_NaN(
    manifest: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> pd.DataFrame:
    '''
    Wenn NaN werte irgendwo in einem report auftauchen wird er ignoriert
    '''

    h5_path = Path(h5_path)

    keep = []
    for _, row in manifest.iterrows():
        key = row[key_col]
        original_name = row.get("original_name", None)

        df = pd.read_hdf(h5_path, key=key)

        # True wenn irgendwo im gesamten Report ein NaN steht (auch
            metric_name etc.)
        has_nan = df.isna().any(axis=None)
        if has_nan:
            warnings.warn(
                f"{key}␣({original_name})␣has␣NaN␣value(s)␣and␣got␣
                    ignored",
                category=UserWarning,
                stacklevel=2,
            )
            continue

        keep.append(row)

    return pd.DataFrame(keep).reset_index(drop=True)

def filter_reports_by_time_skew(
    manifest: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> pd.DataFrame:
    '''
    Wenn der Median des Timewindows eines reports mehr als
        MAX_TIME_SKEW_SECONDS von der Referenzzeit (Median über alle
        Reports) abweicht, wird er ignoriert.
    '''
    h5_path = Path(h5_path)

    # 1) Pro Report eine repräsentative Zeit bestimmen (Median der
        start_time-Spalte)
    per_report_time: list[tuple[int, str, pd.Timestamp, str | None]] = []
    for i, row in enumerate(manifest.itertuples(index=False)):
        key = getattr(row, key_col)
        original_name = getattr(row, "original_name", None)
```

```
    # load report
    df = pd.read_hdf(h5_path, key=key)
    # konvertiere in datetime werte
    t = pd.to_datetime(df[TIME_COL], utc=True, errors="coerce")
    # Berechne median pro report
    per_report_time.append((i, key, t.median(), original_name))

if not per_report_time:
    return manifest.iloc[0:0].copy()

# 2) Referenzzeit: Median über alle Reports
ref = pd.Series([x[2] for x in per_report_time]).median()

# 3) Filter: Abweichung <= MAX_TIME_SKEW_SECONDS
keep_rows = []
for i, key, t_med, original_name in per_report_time:
    skew = abs((t_med - ref).total_seconds())
    if skew > MAX_TIME_SKEW_SECONDS:
        warnings.warn(
            f"{key}␣({original_name})␣time␣skew␣{skew:.1f}s␣>␣{
                MAX_TIME_SKEW_SECONDS}s;␣got␣ignored",
            UserWarning,
            stacklevel=2,
        )
        continue
    keep_rows.append(manifest.iloc[i])

return pd.DataFrame(keep_rows).reset_index(drop=True)
```

Bibliography

[1] B. Boswell, S. Barrett, S. Rajaganapathy, G. Dorai, and M. Qiu. FLARE: Feature-based lightweight aggregation for robust evaluation of IoT intrusion detection. *arXiv preprint arXiv:2504.15375*, 2025. Schlägt FLARE vor, eine leichtgewichtige Feature-Aggregationstechnik für IoT-IDS, die mittels zeitbasierter Sliding Windows statistische Kennzahlen wie Flussrate, Paketgrößen, Inter-Arrival-Zeiten und Entropie aggregiert, um kurzlebige Angriffsmuster wie DoS und DDoS zuverlässig zu erkennen.

[2] CelerData. What is parquet file format?, 2024. Erklärt die Architektur des Parquet-Formats, insbesondere die Vorteile der spaltenorientierten Speicherung und Kompression.

[3] H. Debar and A. Wespi. Aggregation and correlation of intrusion-detection alerts. 2212:85–103, 2001. Beschreibt einen Aggregations- und Korrelationsalgorithmus für Intrusion-Detection-Alerts, der Duplikate und Folgewarnungen erkennt sowie Alerts in Situationen zusammenfasst, um dem Operator eine kondensierte Sicherheitsübersicht zu geben.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. 2019. Führt BERT ein, ein bidirektionales Repräsentationsmodell, das durch Masked Language Modeling (MLM) und Next Sentence Prediction (NSP) tiefen Kontext aus beiden Richtungen gleichzeitig lernt und damit neue Bestleistungen in elf NLP-Aufgaben erzielte[cite: 557, 564, 567, 580, 583].

[5] M. Heigl, E. Weigelt, A. Urmann, D. Fiala, and M. Schramm. Exploiting the outcome of outlier detection for novel attack pattern recognition on streaming data. *Electronics*, 10(17):2160, 2021. Präsentiert das SOAAPR-Framework, das Ausreißererkennungsergebnisse auf Streaming-Daten auswertet und daraus neuartige Angriffsmuster extrahiert.

[6] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. 97, 2019. Stellt den Set Transformer vor, der Self-Attention zur Kodierung von Element-Interaktionen in Mengen nutzt und durch Inducing-Point-Methoden die quadratische Komplexität von Self-Attention auf lineare Komplexität reduziert.

[7] Z. Liu, L. Wang, B. Huang, H. Liu, H. Shi, P. Yang, and H. Dong. Pooling operations in deep learning: From invariable to variable. *BioMed Research International*, 2022. Gibt einen umfassenden Überblick über Pooling-Operationen im Deep Learning, kategorisiert sie nach invariablem und variablem Pooling-Bereich sowie -Kernel, und diskutiert deren Eigenschaften und Einsatzbereiche.

[8] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. 2022. Schlägt E-GraphSAGE vor, ein GNN-basiertes Netzwerk-Intrusion-Detection-System, das sowohl Kanten-Features als auch topologische Informationen aus Netzwerkflussdaten für die Erkennung von Angriffen in IoT-Netzwerken nutzt.

[9] J. Norstad. An introduction to utility theory. 1999. Erläutert die mathematischen Grundlagen von Nutzenfunktionen, insbesondere die Konzepte der Nicht-Sättigung und Risikoaversion[cite: 11, 26]. Das Dokument führt die negative Exponential-Nutzenfunktion zur Modellierung konstanter absoluter Risikoaversion (CARA) ein und zeigt, wie mittels positiver

affiner Transformationen konsistente Präferenzstrukturen skaliert werden können[cite: 112, 385, 412].

[10] PyTorch Contributors. *MultiheadAttention — PyTorch 2.4 Documentation*. Meta AI, 2024. Zugriff am: 13. April 2026.

[11] M. Sölch, A. Akhundov, P. van der Smagt, and J. Bayer. On deep set learning and the choice of aggregations. *arXiv preprint arXiv:1903.07348*, 2019.

[12] R. Taylor-Davies. How good is parquet for wide tables (machine learning workloads) really?, 2023. Analysiert die Performance von Apache Parquet bei Tabellen mit hoher Spaltenanzahl und bewertet den Metadaten-Overhead kritisch.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Führt die Transformer-Architektur ein, die ausschließlich auf Attention-Mechanismen basiert und Rekurrenz sowie Faltungen vollständig ersetzt, und erzielt State-of-the-Art-Ergebnisse bei maschineller Übersetzung.

[14] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. 2018. Präsentiert Graph Attention Networks (GATs), die Masked Self-Attention nutzen, um Knoten-Repräsentationen in Graphen zu lernen, ohne kostspielige Matrixoperationen oder Vorabkenntnis der Graphstruktur zu benötigen[cite: 19, 20].

[15] O. Vinyals, S. Bengio, and M. Kudlur. Order matters: Sequence to sequence for sets. 2016. Zeigt, dass die Reihenfolge von Eingabe- und Ausgabedaten das Lernen signifikant beeinflusst, und schlägt die Read-Process-and-Write-Architektur für permutationsinvariante Mengenverarbeitung sowie eine Suchmethode für optimale Ausgabereihenfolgen vor.

[16] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep sets. 2017. Entwickelt die DeepSets-Architektur, die permutationsinvariante Funktionen auf Mengen durch Summen-Dekomposition charakterisiert und in zahlreichen Aufgaben wie Punktwolken-Klassifikation und Populationsstatistikschätzung angewendet wird.

[17] K. Zhang, F. Zhao, S. Luo, Y. Xin, and H. Zhu. An intrusion action-based IDS alert correlation analysis and prediction framework, 2019. Schlägt das IACF-Framework vor, das Alarme auf Basis intrinsischer Korrelationen zu Intrusion Actions gruppiert und mittels zeitbasierter Sequenzanalyse (TSS) zu Sessions zusammenführt, um mehrstufige Angriffe zu rekonstruieren und vorherzusagen.

Generative KI-Werkzeuge, die in der Arbeit verwendet wurden.

Kapitel	KI-Tool	Version	Anwendung	Erklärung/Kommentar
1.0–4.2	Perplexity AI	2026	Grammatik- und Rechtschreibkorrektion	Der generierte Output wurde ausschließlich zur Überprüfung und Optimierung der Grammatik, Rechtschreibung und Lesbarkeit verwendet. Sämtliche fachlichen Inhalte, Analysen, Interpretationen und wissenschaftliche Schlussfolgerungen stammen vollständig von mir.
1.0–4.2	Gemini AI	2026	Verbesserung der Lesbarkeit	Umformulierung von Sätzen zur besseren Verständlichkeit und Fluss des Textes, ohne inhaltliche Veränderungen.

Eidesstattliche Erklärung

Ich versichere hiermit wahrheitsgemäß, die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, die wörtlich oder inhaltlich übernommenen Stellen als solche kenntlich gemacht und die Satzung der Ostbayerischen Technischen Hochschule Regensburg zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben. Die Arbeit ist weder von mir noch von einer anderen Person an der Ostbayerischen Technischen Hochschule Regensburg oder an einer anderen Hochschule zur Erlangung eines akademischen Grades bereits eingereicht worden.

Regensburg, den 01.04.2026

Yannic Hanel

Anomaly Detection Einheit | Anomalydetectioneinheit | usw.. Detection oder Detektion

immer gleich schreiben

immer gleich schreiben

zu jeder quelle auf welche ich im text verweise

Todo list

Quelle evtl. mit meiner Bachelorarbeit arbeiten 1

Quelle schwierige erkennung wenn netzwerkweite Veränderung und nicht am Knoten selbst 1

Vielleicht finde ich eine begründung warum ich genaue diese events in den hardware report gepackt habe? 2

TODO: Sollte hier wissenschaftlich begründet werden, warum die Wahl auf eine Kombination aus Deep Sets und Attention-Mechanismen gefallen ist? 2

TODO: Die Formel der Utility Theory wurde nicht originär für Gewichtungsfunktionen entwickelt. Wie problematisch ist das? Sollte dies in der Diskussion thematisiert oder direkt eine alternative Methode verwendet werden? 2

TODO: Fokus darauf legen wie die zuvor beschriebenen Methoden in die Architektur einbepflegt wurden 2

Welche Forschungslücke? 2

TODO: Mögliche Forschungslücken: - Untersuchung der Auswirkungen von unterschiedlichen Gewichtungsstrategien Aggregation auf die Qualität der aggregierten Reports - Erweiterung um die LLM gestützte Aufbereitung der Reports für die externe Nutzung - Untersuchung ob der vorgestellte ablauf, erstellen der reports, aggregieren und nutzung der nachbar reports wirklich zu verbesserten Detectionsergebnissen führt (Echo-Chamber-Effekt?). 2

ist das wirklich immer im vergleich oder zeigt mir das die min/max/avg werte innerhalb des fensters an? 12

Deutsch schreiben! 12

verbessere den abschnitt auf Rechtschreibung 13

baue den part noch in die erklärung mit ein. 14

Quelle dafür hinzufügen 23

immer gleich schreiben . . . . . .

immer gleich schreiben . . . . . .

zu jeder quelle auf welche ich im text verweise noch den abschnitt wo es im jeweiligen paper zu fiden ist hinzufügen . . . . . .